Web API

API

Il termine API, acronimo di Application Programming Interface (interfaccia di programmazione delle applicazioni), indica un insieme di definizioni e protocolli per la creazione e l'integrazione di applicazioni software.

Talvolta vengono concepite come una forma di contratto, con una documentazione che rappresenta un accordo tra le parti: se la parte A invia una richiesta remota strutturata in un determinato modo, il software della parte B risponderà in un altro modo determinato.

Le API sono state introdotte agli albori dell'era informatica, molto prima della comparsa dei personal computer. A quell'epoca, un'API era in genere utilizzata come libreria per un sistema operativo. Era a livello locale rispetto al sistema in cui operava, sebbene a volte passasse messaggi tra mainframe. Dopo circa 30 anni, le API sono emerse dai loro ambienti locali. Nei primi anni 2000 si trasformarono in un'importante tecnologia per l'integrazione remota dei dati.

Le API remote sono concepite per interagire attraverso una rete di comunicazione. Si chiamano API remote perché le risorse gestite da tali API si trovano all'esterno del computer che invia la richiesta. Poiché il canale di comunicazione maggiormente utilizzato è Internet, quasi tutte le API vengono progettate in base agli standard web.

Queste ultime utilizzano in genere l'HTTP per richiedere messaggi e fornire una definizione della struttura dei messaggi di risposta. I messaggi di risposta assumono in genere la forma di file XML o JSON, due formati diffusi perché presentano i dati in modo da consentire alle altre app di gestirli facilmente.

Web API (Web service vs Web API)

L'utilizzo del Web come piattaforma di programmazione è un'idea che si è diffusa intorno al 2000 con l'introduzione dei Web service, cioè di un meccanismo per l'interoperabilità tra applicazioni basato sulle tecnologie e gli standard definiti per il Web.

I Web service consentono di far interagire due applicazioni indipendentemente dal sistema operativo su cui girano e dal linguaggio di programmazione utilizzato. Tecnicamente il meccanismo di interazione è basato sul protocollo SOAP, su un insieme di regole che definiscono la struttura dei messaggi scambiati dalle applicazioni e tutta una serie di accorgimenti per garantire sicurezza, affidabilità, corretta gestione degli eventi, ecc. L'approccio è sostanzialmente basato su una definizione di funzione richiamabile via Web.

Un approccio alternativo ai Web service basati su SOAP, che è stato proposto negli stessi anni ma è stato riscoperto soltanto negli ultimi tempi, è l'approccio noto come REST (Representational State Transfer) e basato su una serie di principi che un'applicazione deve rispettare. In sintesi, nella visione RESTful un Web service non definisce una funzione richiamabile da remoto ma mette a disposizone delle risorse su cui è possibile effettuare le classiche operazioni CRUD sfruttando i metodi del protocollo HTTP.

Due visioni diverse del Web come piattaforma di elaborazione che sempre più spesso vengono distinte parlando semplicemente di Web service quando si fa riferimento a quelli basati si SOAP e di Web API quando ci ispira al modello REST.

Web API (SOAP vs REST)

Vista la continua diffusione delle API web, è stato sviluppato un protocollo specifico con lo scopo di standardizzare lo scambio delle informazioni, denominato Simple Object Access Protocol, o protocollo SOAP. Le API progettate con il protocollo SOAP usano il linguaggio XML come formato del messaggio e ricevono le richieste tramite HTTP o SMTP. Il protocollo SOAP facilita la condivisione delle informazioni per le app eseguite su ambienti diversi o scritte in linguaggi differenti.

È stata introdotta anche la specifica Representational State Transfer (REST). Le API web che rispettano i vincoli architetturali REST vengono definite API RESTful.

La differenza tra REST e SOAP è sostanziale: il SOAP è un protocollo, mentre il REST è un tipo di architettura e ciò implica l'assenza di uno standard ufficiale per le API web RESTful.

Web API (API REST)

Le API sono definibili RESTful se rispettano i sei vincoli di un sistema RESTful:

- Architettura client-server: l'architettura REST è costituita da client, server e risorse e gestisce le richieste tramite il protocollo HTTP.
- Stateless: nessun contenuto client è archiviato nel server tra le richieste. Le informazioni relative allo stato della sessione sono invece contenute nel client.
- Supporto cache: il caching può eliminare la necessità di alcune interazioni client-server.
- Sistema a livelli: le interazioni client-server possono essere mediate da livelli aggiuntivi, i quali possono offrire altre funzionalità, come bilanciamento del carico, condivisione della cache o sicurezza.
- Codice on demand (opzionale): i server possono ampliare la funzionalità di un client trasferendo del codice eseguibile.
- Interfaccia uniforme: è il vincolo principale per la progettazione di API RESTful e prevede 4 aspetti:
    1. Identificazione delle risorse nelle richieste: le risorse vengono identificate nelle richieste e vengono distinte dalle rappresentazioni restituite al client.
    2. Manipolazione delle risorse tramite le rappresentazioni: i client ricevono file che rappresentano le risorse e che devono contenere le informazioni necessarie per consentirne la modifica o l'eliminazione.
    3. Messaggi autodescrittivi: ogni messaggio restituito a un client contiene le informazioni necessarie per descrivere come il client deve elaborare l'informazione.
    4. Ipermedia come motore dello stato dell'applicazione: accedendo alla risorsa, il client REST deve poter individuare, attraverso hyperlink, tutte le altre azioni disponibili al momento.

Benché sembrino numerosi, questi vincoli sono molto più semplici rispetto a un protocollo prescritto e, proprio per questo, le API RESTful sono molto più diffuse dei metodi SOAP.

Negli ultimi anni, la specifica OpenAPI si è imposta come standard comune per la definizione delle API REST, poiché consente agli sviluppatori di realizzare interfacce API REST indipendenti dal linguaggio e facilmente comprensibili agli utenti.

Web API

API Web di base ASP.NET

ASP.NET Core è un framework open source, multipiattaforma e ad alte prestazioni per la creazione di app moderne, abilitate al cloud e connesse a Internet.

Con ASP.NET Core si può:

- Creare app e servizi web, app per backend per dispositivi mobili
- Utilizzare strumenti di sviluppo su Windows, macOS e Linux
- Distribuire sul cloud o in locale
- Eseguire l'app su .NET

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Web API (openAPI)

Supporto openAPI nelle app per le API principali di ASP.NET

La specifica OpenAPI è uno standard indipendente dal linguaggio di programmazione per la documentazione delle API HTTP. Questo standard è supportato in API minime tramite una combinazione di API predefinite e librerie open source. L'integrazione OpenAPI in un'applicazione presenta tre aspetti chiave:

- Generazione di informazioni sugli endpoint nell'app.
- Raccolta delle informazioni in un formato corrispondente allo schema OpenAPI.
- Esposizione dello schema OpenAPI generato tramite un'interfaccia utente visiva o un file serializzato.

Esercitazione7 – Web API

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Verificare gli SDK .NET installati: **`dotnet --list-sdks`**
- Creare un nuovo progetto .NET 8 di tipo web API denominato Esercitazione7
  - ~~`dotnet new webapi -n 7-DemoApi8 -f net8.0`~~ **il trattino è sbagliato!!**

```
PS C:\Dev\CorsoITS\MyExercises> dotnet new webapi -n 7-demoApi8 -f net8.0
>>
Errore: opzioni non valide:
-n
   '-n' non è un'opzione valida
7-demoApi8
   '7-demoApi8' non è un'opzione valida
-f
   '-f' non è un'opzione valida
net8.0
   'net8.0' non è un'opzione valida

Per altre informazioni, eseguire:
   dotnet new webapi -h

Per informazioni dettagliate sul codice di uscita, vedere https://aka.ms/templating-exit-codes#127
PS C:\Dev\CorsoITS\MyExercises> dotnet new webapi -n 7-demoApi8 -f net8.0
>>
Creazione del modello "API Web ASP.NET Core" completata.
```

  - `dotnet new webapi -n 7-DemoApi8 -f net8.0`

Esercitazione7 – Web API

```
var builder =
WebApplication.CreateBuilder(args);
// Add services to the container.
// Learn more about configuring Swagger/OpenAPI
at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{   app.UseSwagger();
//http://localhost:5280/swagger/index.html
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

var summaries = new[]
{
    "Freezing", "Bracing", "Chilly", "Cool",
"Mild", "Warm", "Balmy", "Hot", "Sweltering",
"Scorching«
};

app.MapGet("/weatherforecast", () =>{
    var forecast =  Enumerable.Range(1,
5).Select(index =>
        new WeatherForecast
        (
DateOnly.FromDateTime(DateTime.Now.AddDays(inde
x)),
        Random.Shared.Next(-20, 55),

summaries[Random.Shared.Next(summaries.Length)]
        )).ToArray();
    return forecast;
})
.WithName("GetWeatherForecast")
.WithOpenApi();


app.Run();

record WeatherForecast(DateOnly Date, int
TemperatureC, string? Summary)

{

    public int TemperatureF => 32 +
(int)(TemperatureC / 0.5556);

}
```

Program.cs

Le API minime offrono il supporto predefinito per la generazione di informazioni sugli endpoint in un'app tramite il pacchetto Microsoft.AspNetCore.OpenApi.

- AddEndpointsApiExplorer : configura l'app per l'uso di Esplora API per individuare e descrivere gli endpoint con annotazioni predefinite.
- WithOpenApi esegue l'override delle annotazioni predefinite generate da Esplora API con quelle generate dal pacchetto Microsoft.AspNetCore.OpenApi.
- UseSwagger aggiunge il middleware Swagger.
- 'UseSwaggerUI' abilita una versione incorporata dello strumento dell'interfaccia utente di Swagger.
- WithNameIEndpointNameMetadata: nell'endpoint viene usato per la generazione di collegamenti e viene considerato come l'ID operazione nella specifica OpenAPI dell'endpoint specificato.

Esercitazione7 – Web API

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi"
Version="8.0.8" />
    <PackageReference Include="Swashbuckle.AspNetCore"
Version="6.4.0" />
  </ItemGroup>

</Project>
```

Esercitazione7.csproj
Microsoft.AspNetCore.OpenApi viene aggiunto come PackageReference a un file di progetto.

Esercitazione7 – Web API

GET, POST, PUT e DELETE nella progettazione di API RESTful

Metodi HTTP:

- GET : Utilizzato per recuperare dati da un server.
- POST : Utilizzato per inviare dati al server per creare una nuova risorsa.
- PUT : utilizzato per aggiornare o sostituire una risorsa esistente.
- DELETE : Utilizzato per rimuovere una risorsa dal server.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione7 – Web API

```
app.MapGet("/weatherforecast", () =>
{
    var forecast =  Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast
        (
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)]
        ))
        .ToArray();
    return forecast;
})
.WithName("GetWeatherForecast")
.WithOpenApi();
```

Program.cs – Minimal API
Sono un approccio semplificato per la creazione di API HTTP veloci con ASP.NET Core. È possibile creare endpoint REST completamente funzionanti con codice e configurazione minimi. Salta l'impalcatura tradizionale ed evita controller non necessari dichiarando in modo fluido percorsi e azioni API.

Un API endpoint è un luogo digitale esposto tramite l'API dal quale l'API riceve le richieste e invia le risposte. Ogni endpoint è un URL (Uniform Resource Locator) che fornisce la posizione di una risorsa sul server dell'API.

Un'API è un insieme di protocolli e strumenti per facilitare l'interazione tra due applicazioni. Un endpoint è un posto sull'API dove avviene lo scambio. Gli endpoint sono URI (Uniform Resource Identifiers) su un'API a cui un'applicazione può accedere.
All'endpoint, l'API accederà alle risorse di cui ha bisogno da un server per eseguire un'attività specifica, come il recupero di determinati dati o informazioni.

Esercitazione7 – Web API



COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione7 – Web API (.NET 8)

- Compilare e Consumare Web API
  - `dotnet build`
  - `dotnet run`
  - `…e poi???`
  - `vedi slide successiva >>>`
- Controllare la configurazione dell'applicazione (`launchSettings.json`)

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE TURISMO E NUOVE TECNOLOGIE

Esercitazione7 – Web API

Avviare il progetto

Nel terminale apparirà un indirizzo per testare la web api con Swagger:

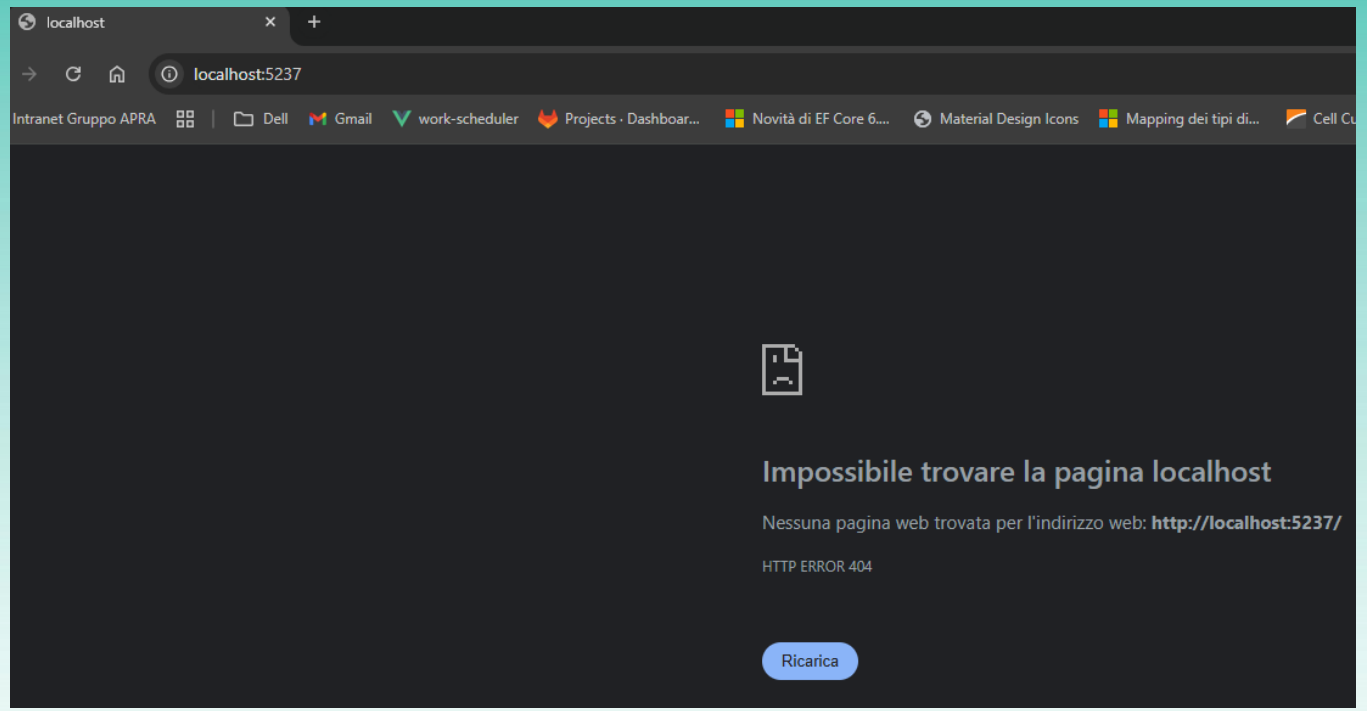
```
PS C:\Users\b.pierangeli\Desktop\CorsoITS\Esercitazione7> dotnet run
Compilazione...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5237
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\b.pierangeli\Desktop\CorsoITS\Esercitazione7
```

Posizionarsi sopra l'url e CTRL+click con il tasto sinistro del mouse.

Si aprirà una tab nel browser, come nella slides seguente

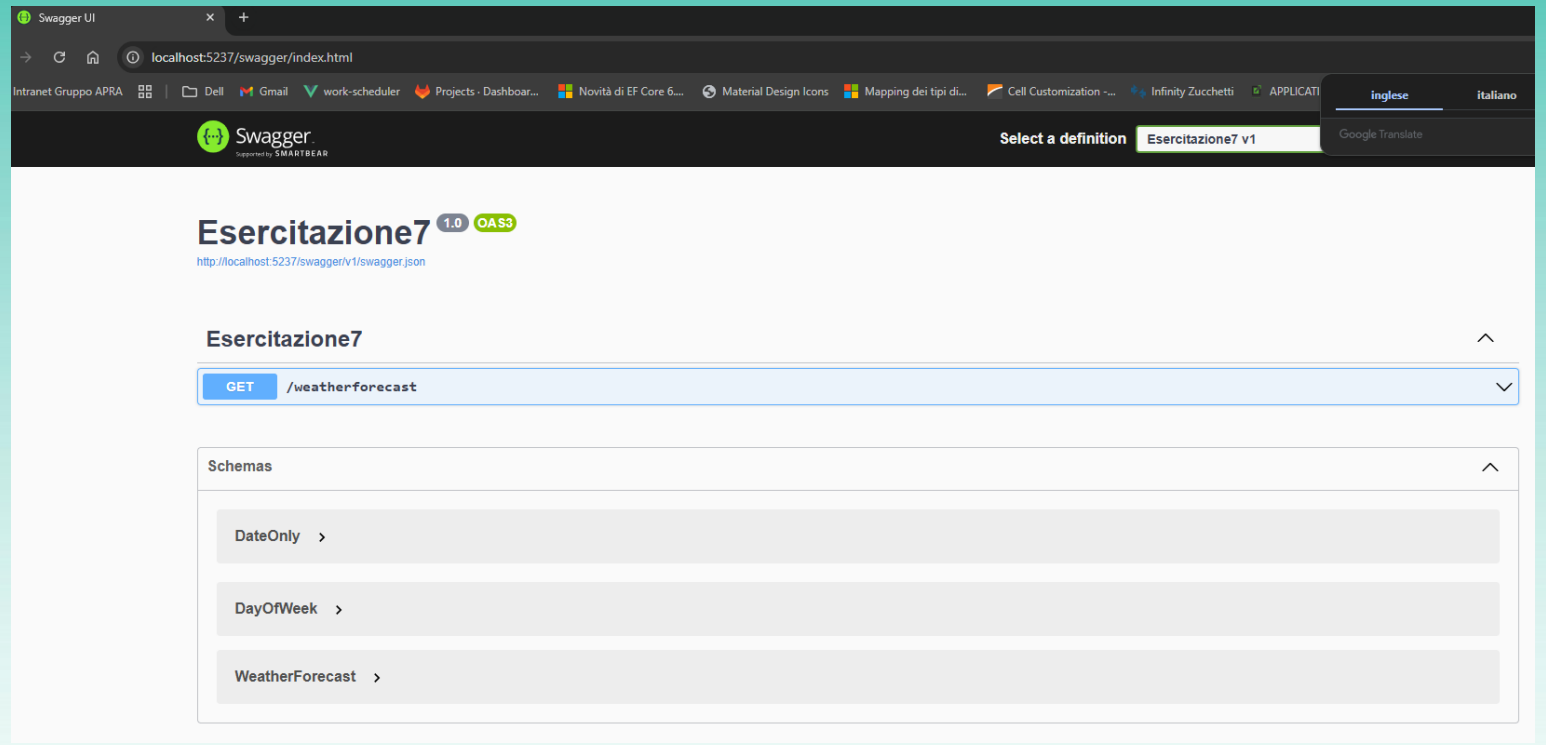
Per stoppare l'esecuzione del progetto API CTRL+C

Esercitazione7 – Web API



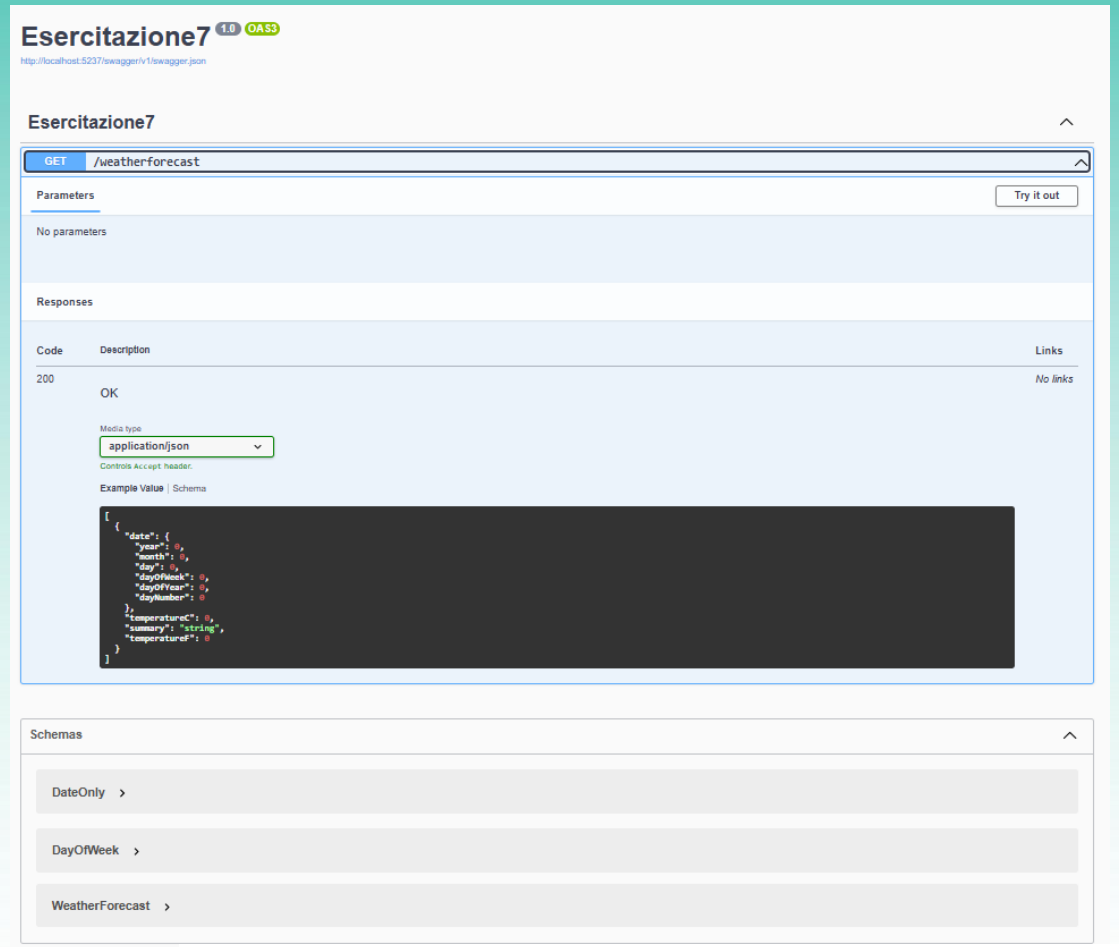
Aggiungere /swagger all'url e cliccare invio. Vedi slide successiva

Esercitazione7 – Web API



Quella che appare è l'interfaccia grafica di swagger, da cui possiamo testare le api del progetto.

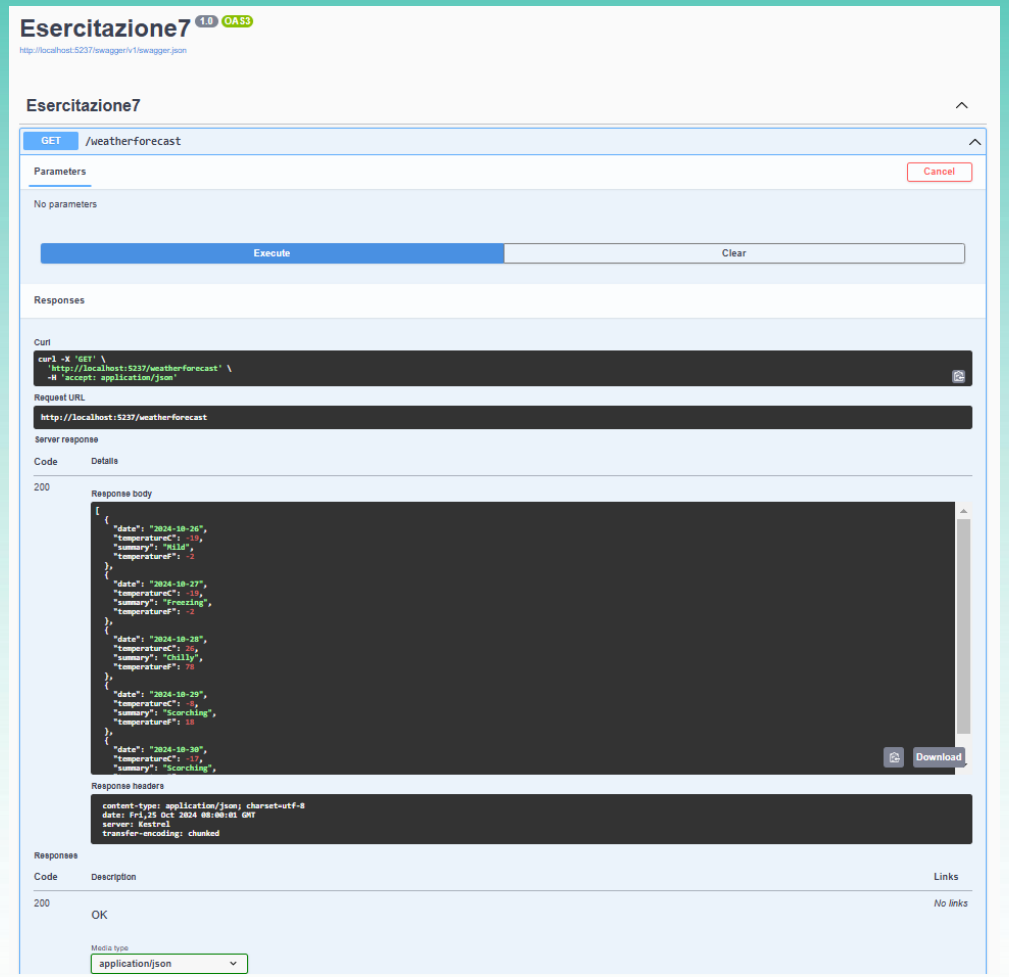
Esercitazione7 – Web API



Vediamo:

1. Il nome del progetto (Esercitazione7);
2. La chiamata HTTP GET e l'endpoint (/weatherforecast);
3. Parameters, se presenti (in questo caso No parameters);
4. Responses, sono le risposte accettate, in questo caso:
   - Status CODE: 200
   - Tipo della risposta: application/json.

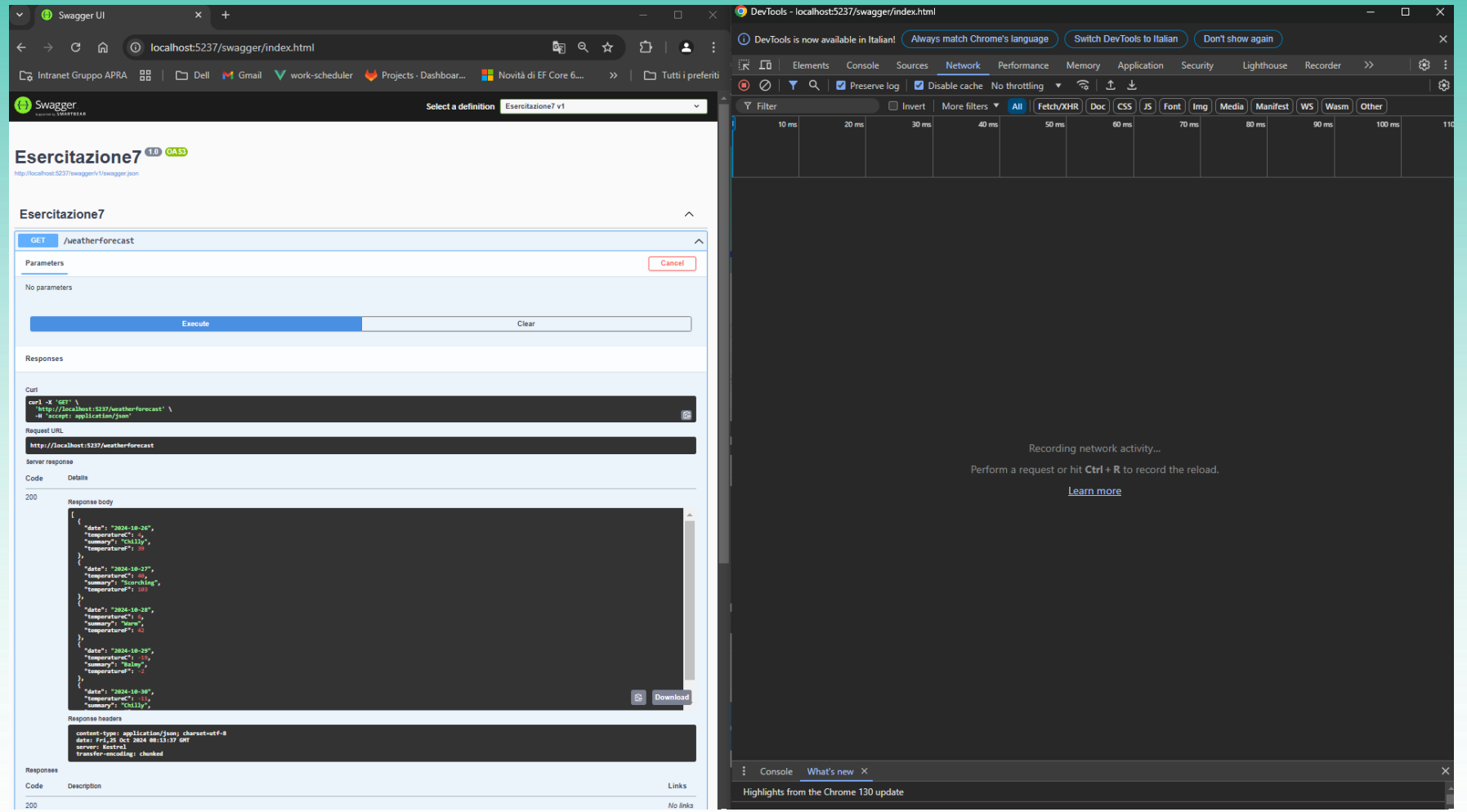
Esercitazione7 – Web API



Per eseguire, cliccare su Try it out e Execute.
Otteniamo la risposta (Response body).

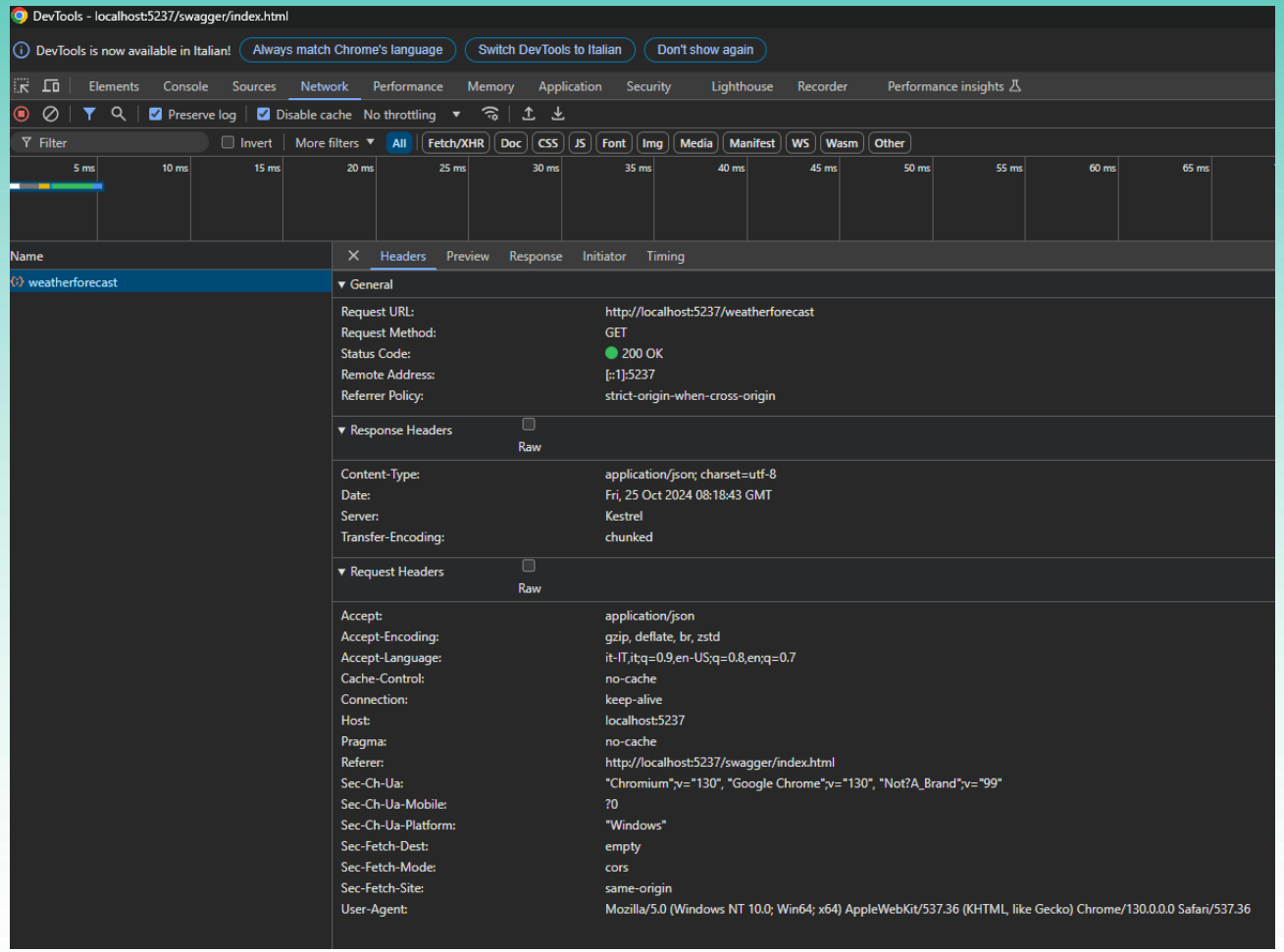
Ora clicchiamo F12 per aprire gli strumenti dello sviluppatore e posizioniamoci nella tab Network, come nella slide successiva

Esercitazione7 – Web API



Se da qui ora clicchiamo su Execute, vediamo che la tab Network si popola con una riga, se clicchiamo in quella riga vediamo tutte le informazioni relative alla chiamata HTTP, come nella slide successiva

Esercitazione7 – Web API



Nella tab:

- Headers, ritroviamo le informazioni generali, con i dettagli della chiamata: l'URL(Request URL), il tipo di chiamata(Request Method), lo status code della risposta e il tipo di risposta accettata(Accept);
- Preview o Response, ritroviamo i dati inviati dal server come risposta, se presenti;
- Se avessimo avuto dei parametri nel body avremmo visualizzato anche una ulteriore tab (Payload) con i dati inviati dal client al server.

Esercitazione7.1 – Web API (.NET 10)

- Creare un nuovo progetto, come prima ma su .NET 10
  - `dotnet new webapi -n 7.1-DemoApi10 -f net10.0`
- Ed eseguire quanto fatto in precedenza per consumare API
  - Che cosa succede?

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

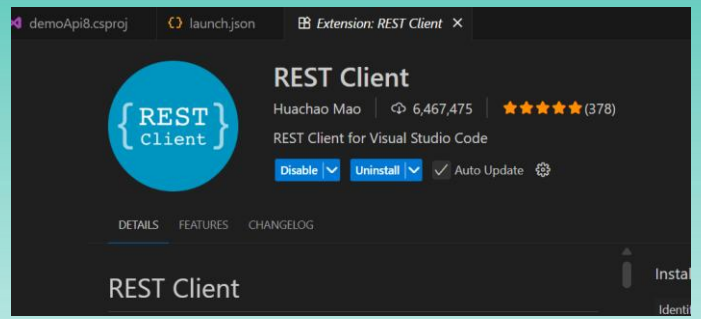
Esercitazione7 – Web API (.NET 10)

- Creare un nuovo progetto, come prima ma su .NET 10
  - `dotnet new webapi -n 7demoApi10 -f net10.0`
- Ed eseguire quanto fatto in precedenza per consumare API
  - Che cosa succede? https://medium.com/@dev.mjdhanesh/scalar-api-reference-in-net-9-a-modern-alternative-to-swagger-cf2d48cdd258
  - Aggiungere e configurare Scalar
- Utilizzare un client REST esterno:
  - REST client (VS code ext)
  - Postman,
  - Bruno

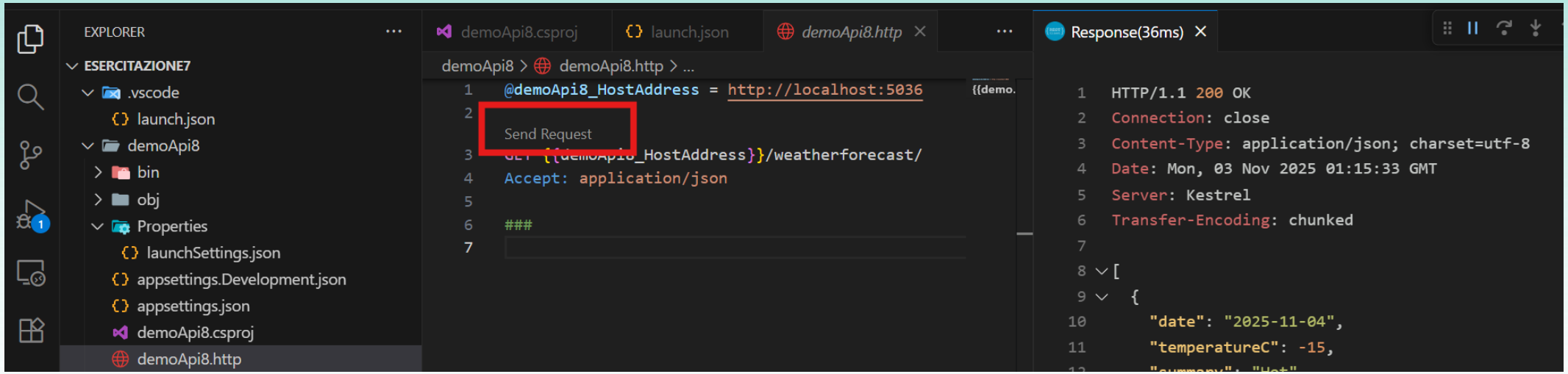




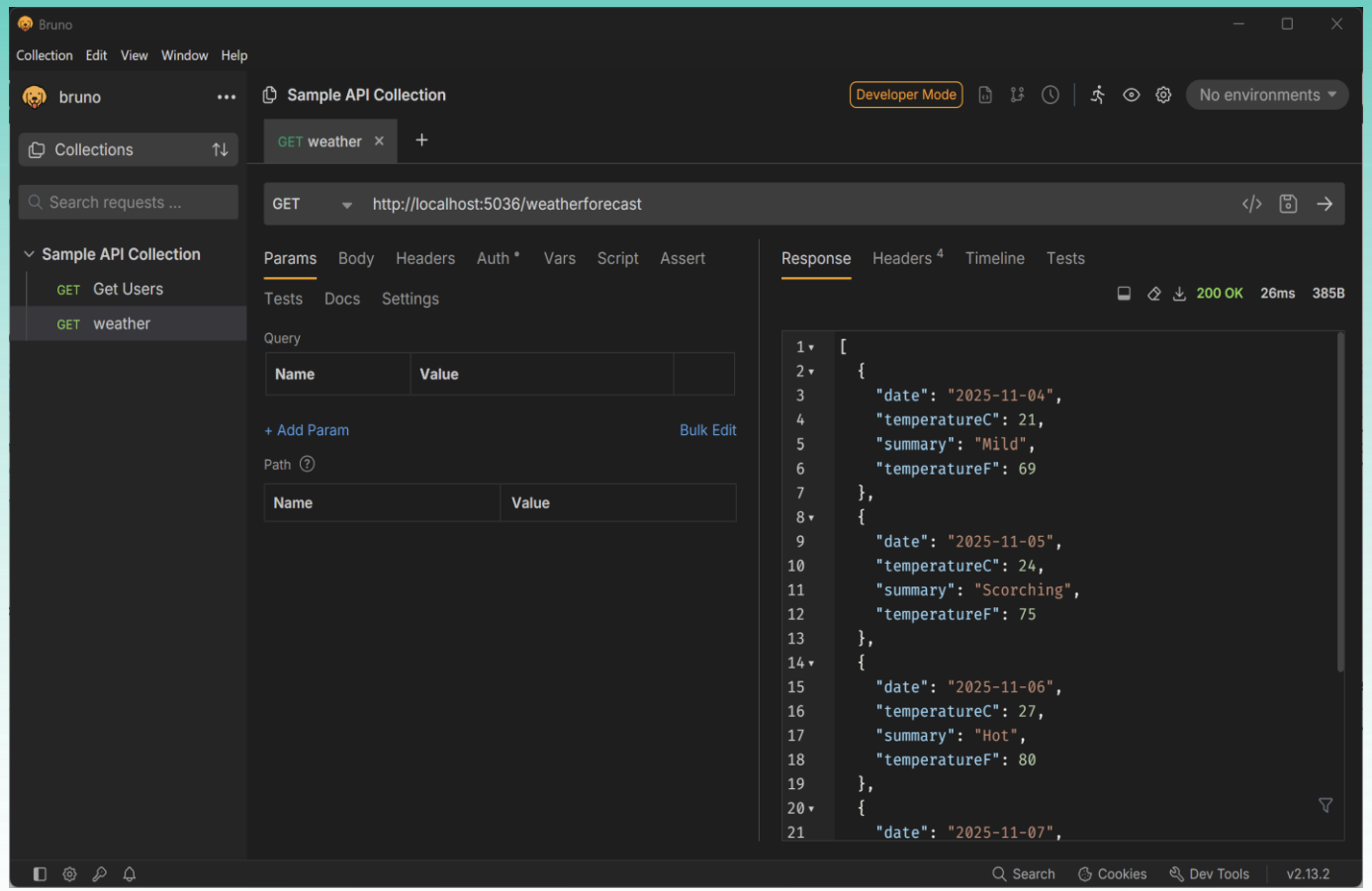
Esercitazione7 – Web API (REST Client)



- Ha come vantaggio la possibilità di sfruttare al volo le richieste .http nella codebase
- Come testare?



Esercitazione7 – Web API (Bruno)



Perché Bruno e non Postman?

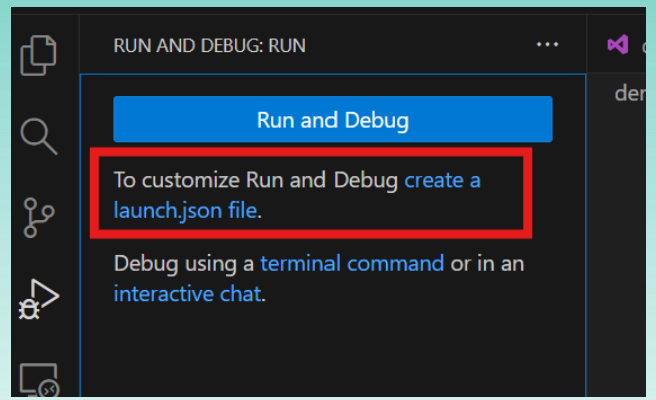
Bruno

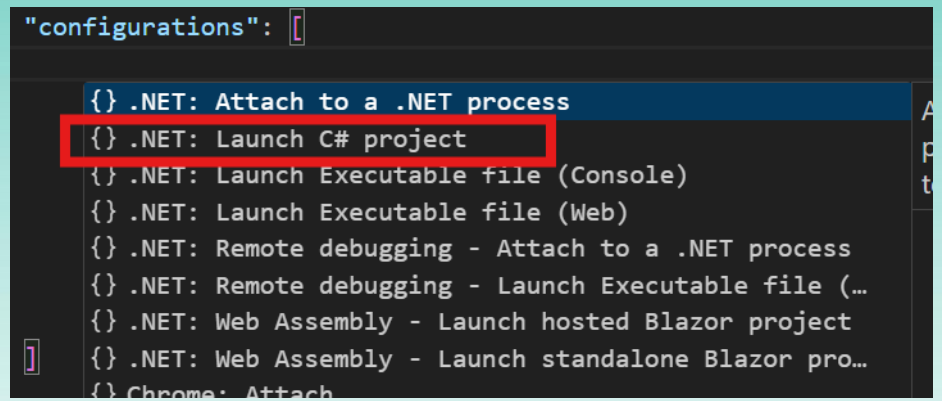
https://www.usebruno.com/compare/bruno-vs-postman

Bruno semplifica il test delle API grazie a un'interfaccia grafica, gestione di collezioni e variabili, e strumenti avanzati che migliorano la produttività rispetto a client REST più semplici.

Esercitazione7 – Launcher VS Code

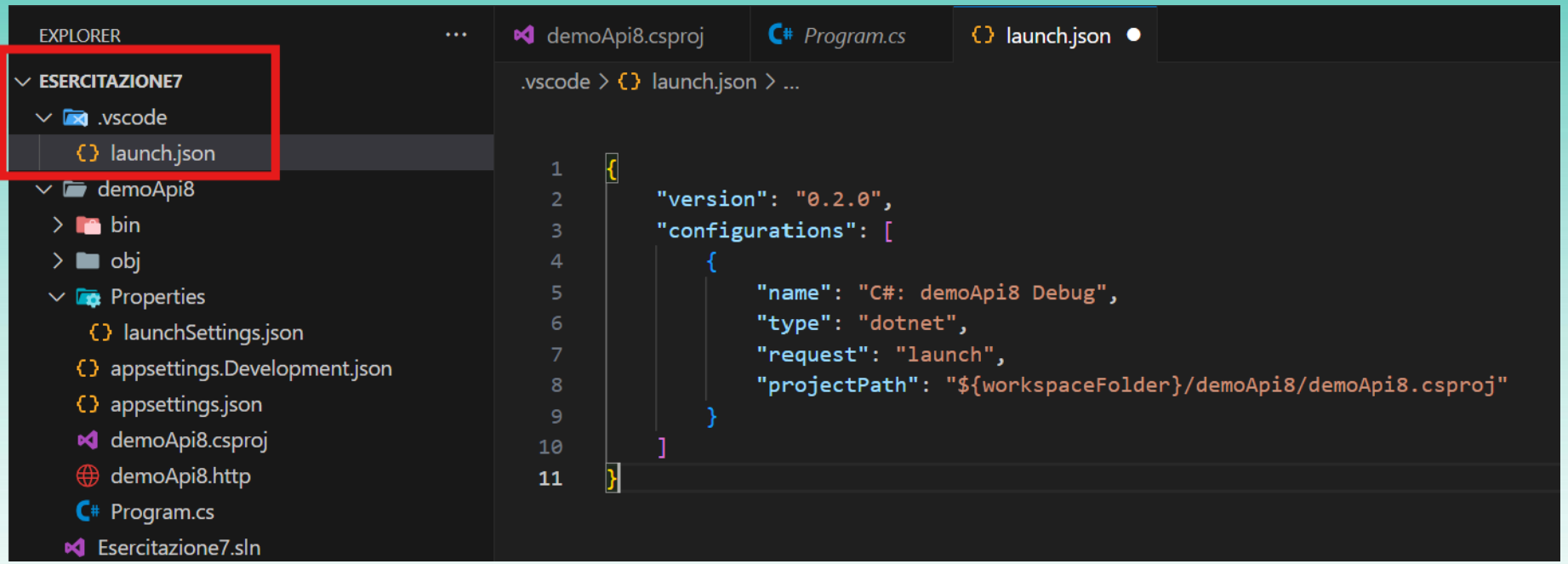
- Creare una configurazione per il debug di VS code (tipo debugger, progetto di avvio, gli argomenti, variabili di ambiente)
- Selezionare C#






```
// For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
"version": "0.2.0",
"configurations": [
    {
        "name": "C#: <project-name> Debug",
        "type": "dotnet",
        "request": "launch",
        "projectPath": "${workspaceFolder}/<relative-path-to-project-folder><project-name>.csproj"
    }
```

Esercitazione7 – Launcher VS Code



Provare da scorciatoia F5 (o FN+F5 a seconda dell'impostazione del bios)

Esercitazione7 – Launcher VS Code

E per lanciare entrambi i progetti insieme?

Esercitazione7 – Launcher VS Code

….provatelo a casa!!!

Suggerimento 👇
https://learn.microsoft.com/en-us/microsoft-edge/visual-studio-code/microsoft-edge-devtools-extension/launch-json

Esercitazione8 – Web API Person

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo web API denominato Esercitazione8
  - `dotnet new webapi -n Esercitazione8`

Esercitazione8 – Web API Person

```
public class Person
{
    public int id { get; set; }
    public string? Name { get; set; }
    public int Age { get; set; }
}
```

Person.cs
Creiamo una classe Person come in figura

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione8 – Web API Person

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
```

Program.cs
Personalizziamo ora il nostro Program, sempre utilizzando l'interfaccia grafica di Swagger, ma utilizzando le nostre API.

A fianco vediamo la prima parte del codice, essenzialmente uguale al Program.cs che avevamo nella precedente Esercitazione.

Esercitazione8 – Web API Person

```
Dictionary<int, Person> dict = new Dictionary<int, Person>();
app.MapGet("/getPeople", () =>
{
    if(dict.Count > 0)
        return dict.Values.ToArray();
    else
        return [];
});
app.MapGet("/getPerson/{id}", (int id) =>
{
    if(dict.Count > 0 && dict.ContainsKey(id))
        return dict[id];
    else
        return null;
});
```

Program.cs

Utilizziamo un Dictionary di tipo chiave-valore per simulare una tabella del DB (la chiave corrisponde all'id della persona, i valori a Person), ogni elemento di dict corrisponde a una persona con chiave univoca id.

Inseriamo poi le API:

- getPeople chiamata GET senza parametri che controlla se dict contiene degli elementi, in tal caso restituisce un array di tutte i valori di dict (tutte le Person sotto forma di array), altrimenti restituisce un array vuoto;
- getPerson/id chiamata GET con parametro id (nell'url) che controlla che dict contiene degli elementi e che contiene la chiave id, in tal caso restituisce un valore Person di dict con chiave uguale a id (dict[id]), altrimenti restituisce un array null;

Esercitazione8 – Web API Person

```
app.MapPost("/create", (Person person) =>
{
    if (dict.Count > 0 && dict.ContainsKey(person.id))
        return Results.BadRequest();
    else
    {
        dict.Add(person.id, person);
        return Results.Ok();
    }
});
app.MapPut("/update", (Person person) =>
{
    if ((dict.Count > 0 && !dict.ContainsKey(person.id)) || dict.Count == 0)
        return Results.NotFound();
    else
    {
        dict[person.id] = person;
        return Results.Ok();
    }
});
```

Program.cs

- Create chiamata POST con parametro di tipo Person (nel body) che controlla se dict contiene elementi e che l'id del parametro person è presente come chiave in dict, in tal caso restituisce BadRequest, altrimenti inserisce un nuovo elemento in dict (con chiave person.id e valore person) e restituisce OK.
- Update chiamata PUT con parametro di tipo Person (nel body) che controlla se dict contiene elementi e che l'id di person non è presente come chiave in dict oppure controlla che dict non contiene elementi, in entrambi i casi restituisce NotFound (quindi è sufficiente che uno dei due casi si verifichi), altrimenti modifica l'elemento di dict con chiave uguale a person.id con i nuovi valori del parametro person e restituisce OK;

Esercitazione8 – Web API Person

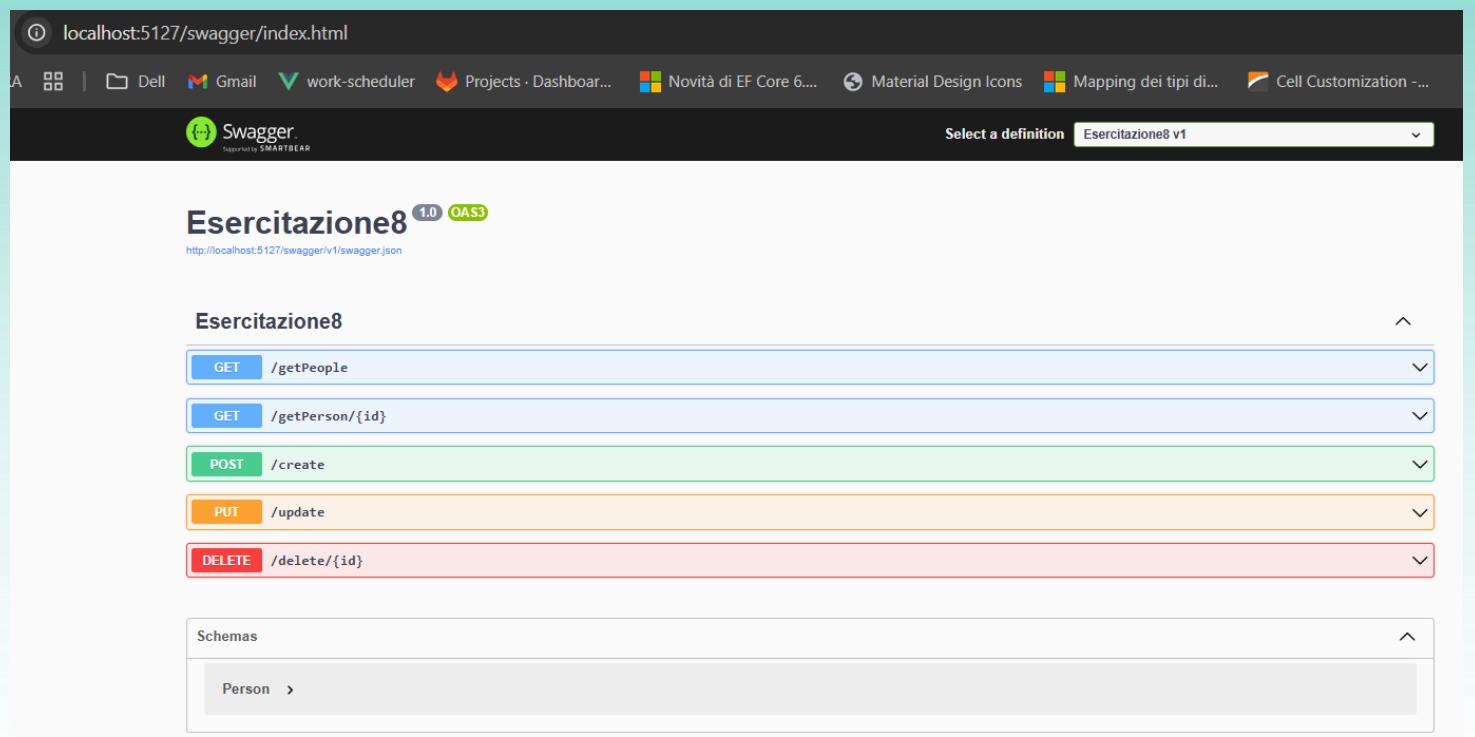
```
app.MapDelete("/delete/{id}", (int id) =>
{
    if ((dict.Count > 0 && !dict.ContainsKey(id)) ||
dict.Count == 0)
        return Results.NotFound();
    else
    {
        dict.Remove(id);
        return Results.Ok();
    }
});
app.Run();
```

Program.cs

- Delete/id chiamata DELETE con parametro id (nell'url) che controlla se dict contiene elementi e che l'id non è presente come chiave in dict oppure che dict non contiene elementi, in entrambi i casi restituisce NotFound (quindi è sufficiente che uno dei due casi si verifichi), altrimenti elimina l'elemento di dict con chiave id e restituisce OK.

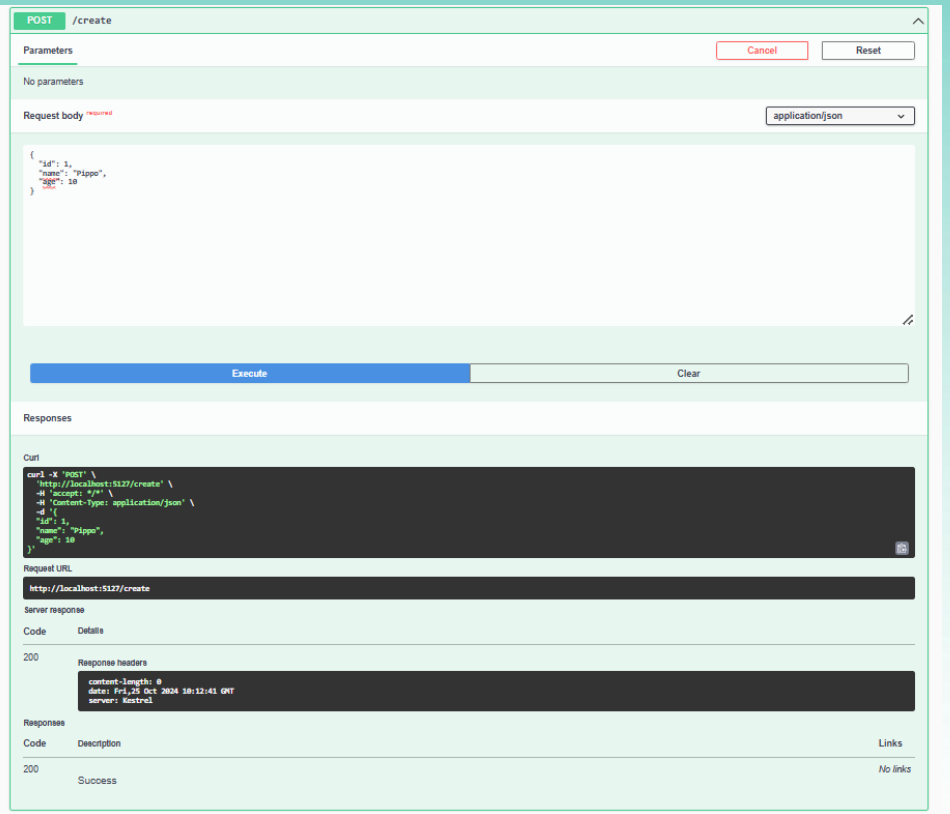
Esercitazione8 – Web API Person

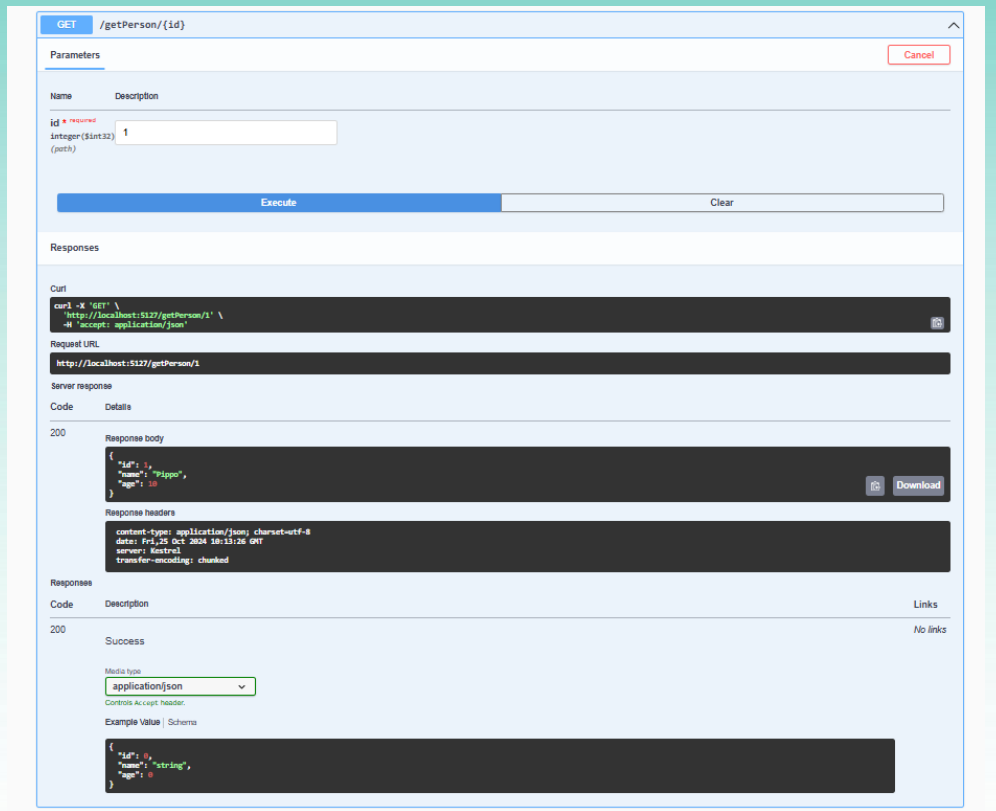
Avviando l'app da terminale con il comando
dotnet run
e aprendo Swagger nel browser possiamo testare le api:



Esercitazione8 – Web API Person

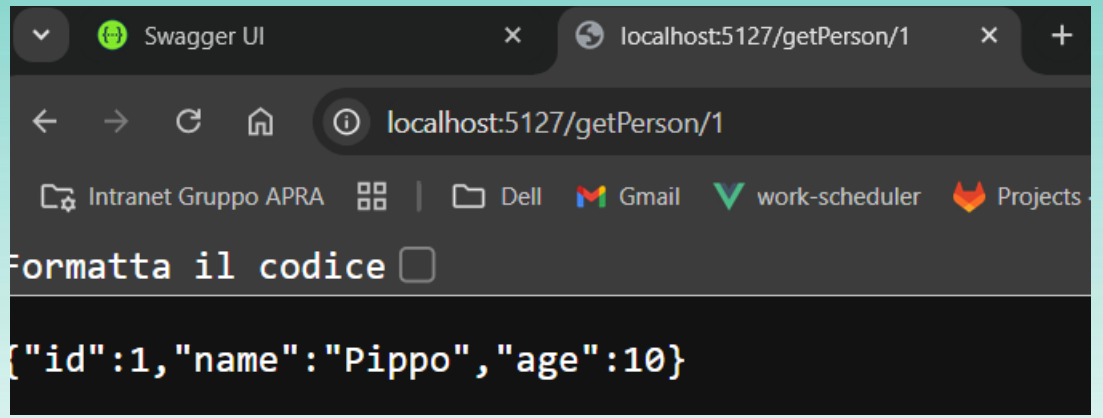
Esempio di inserimento di una persona e successiva lettura tramite id (notare che la POST passa un json come parametro, la GET passa il parametro nell'url):





Esercitazione8 – Web API Person

E’ possibile fare la stessa chiamata precedente anche in una nuova scheda del browser come in seguito:



Quindi non ci stiamo interfacciando con Swagger, ma stiamo richiamando direttamente l’endpoint getPerson con parametro id = 1

Esercitazione9 – CSR

Cerchiamo di dare un’architettura pulita alla nostra applicazione.

Quindi partendo dall’Esercitazione8, la nuova applicazione avrà le stesse funzionalità precedenti, ma la struttura del progetto verrà modificata in modo da rispettare la CLEAN ARCHITECTURE.

Oggi nel mondo del software è molto diffuso l’uso delle cosiddette Clean Architecture. Vengono chiamate così perché si basano tutte sullo stesso principio di progettazione del software: la separazione delle responsabilità.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione9 – CSR

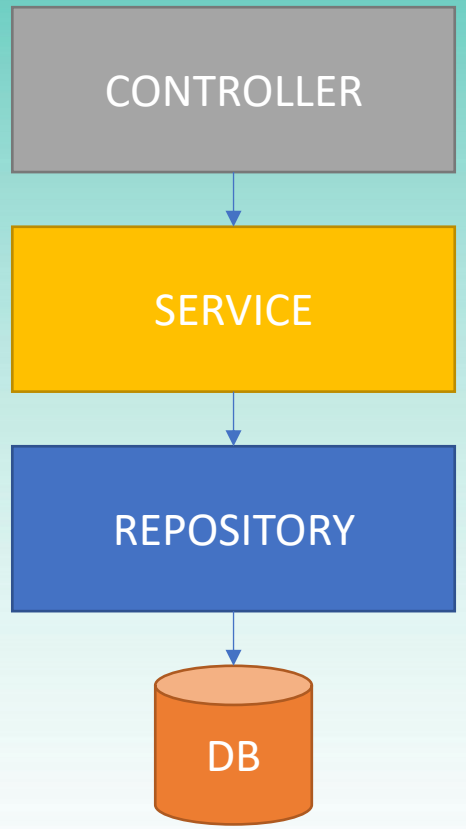
MODELLO CSR

Nello sviluppo software, creare applicazioni scalabili, manutenibili e testabili è fondamentale.

Un modello architettonico che aiuta a raggiungere questi obiettivi è il modello Controller-Service-Repository (CSR). Questo modello promuove la separazione delle preoccupazioni dividendo la logica dell'applicazione in tre livelli distinti: il Controller, il Service e il Repository.

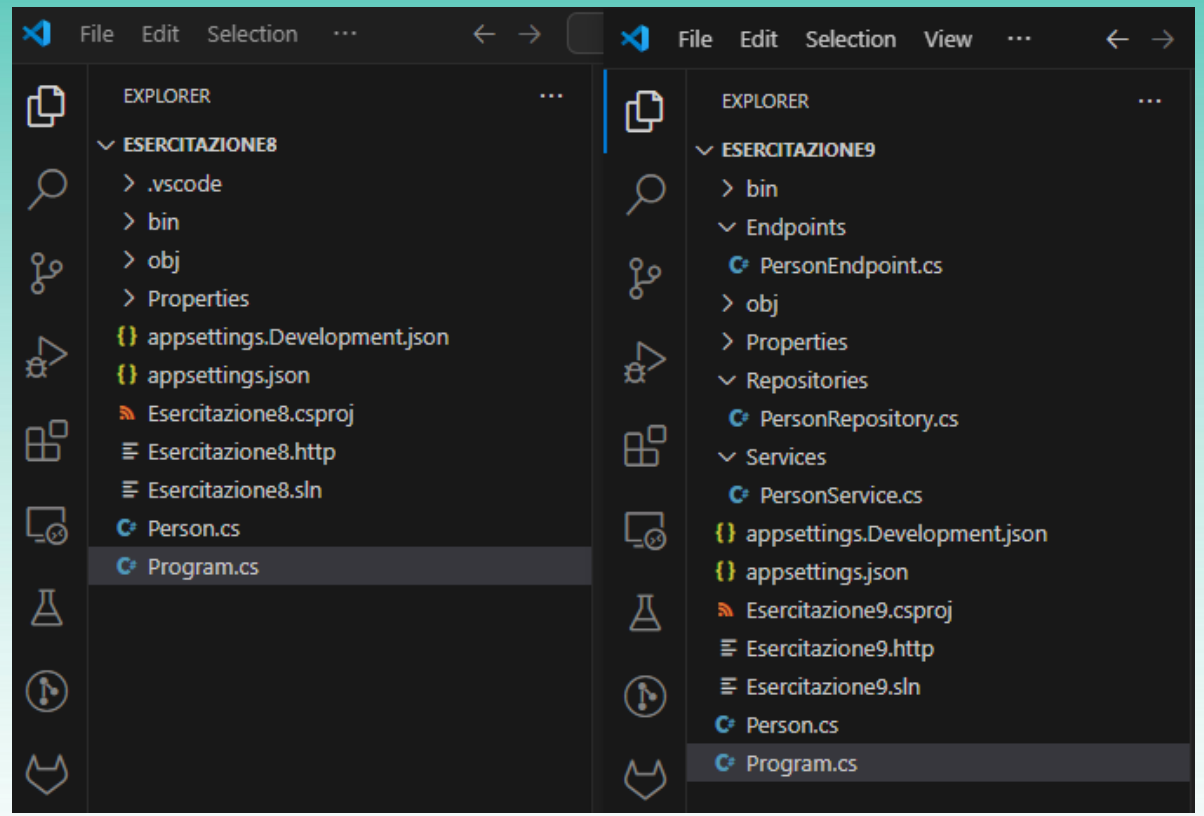
Ogni livello ha una responsabilità specifica, rendendo il codice più facile da gestire, testare e scalare:

- il livello Controller gestisce le richieste HTTP in arrivo, le elabora e restituisce le risposte HTTP appropriate. Agisce come intermediario tra il client e il livello Service;
- il livello Service contiene la logica aziendale dell'applicazione. Elabora i dati ricevuti dal controller, esegue le operazioni necessarie e comunica con il livello di Repository per recuperare o rendere persistenti i dati;
- il livello Repository, gestisce l'accesso ai dati e la persistenza. Comunica con il database o altre fonti di dati, incapsulando la logica per interrogare, salvare e aggiornare i dati.



COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione9 – CSR



Differenze fra Esercitazione8 e Esercitazione9

Mettiamo a confronto l'architettura delle due applicazioni:

- Nell'Esercitazione8 avevamo creato una solo classe Person all'interno del file Person.cs e endpoint, logica dell'applicazione e accesso ai dati avvenivano all'interno del Program.cs;
- Nell'Esercitazione9 sono invece presenti altre cartelle che permettono di rispettare la separazione delle responsabilità:
    1. Il livello Controller lo troviamo all'interno del file PersonEndpoint.cs della cartella Endpoints;
    2. Il livello Service lo troviamo all'interno del file PersonService.cs della cartella Services;
    3. Il livello Repository lo troviamo all'interno del file PersonRepository.cs della cartella Repositories.

Le cartelle Endpoints, Services e Repositories ci permettono di suddividere le responsabilità e di farlo per ogni singolo modello (classe).
Quindi, se oltre alla classe Person di Person.cs in futuro aggiungeremo un'altra classe (ad esempio la classe Animal in un file Animal.cs), aggiungeremo altri file nelle cartelle Endpoints, Services e Repositories: esattamente un ulteriore file per ogni livello di responsabilità, cioè per ogni cartella (rispettivamente i files AnimalEndpoint.cs, AnimalService.cs e AnimalRepository.cs).

Esercitazione9 – CSR

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

var app = builder.Build();

app.MapAPIEndpoints();

// Configure the HTTP request pipeline.

if (app.Environment.IsDevelopment())

{

    app.UseSwagger();

    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.Run();
```

Program.cs

Rispetto all'Esercitazione8, nel Program.cs, togliamo tutto il codice che appartiene ai livelli di responsabilità CSR, per spostarli in altri files.
Ricordiamo però che il Program.cs è il punto di avvio dell'applicazione ed è quello che si occupa di creare ed eseguire l'app.
E' quindi necessario dire all'app di aggiungere le nostre API, altrimenti non verrano create.
Per farlo richiamiamo un metodo, statico, creato da noi: MapAPIEndpoints() che contiene la definizione delle API.
Vediamo con la slide successiva la definizione del metodo MapAPIEndpoints() della classe PersonEndpoints.

Esercitazione9 – CSR

```
public static class PersonEndpoints

{

    private static PersonService personService = new
PersonService();

    public static void MapAPIEndpoints(this
IEndpointRouteBuilder app)

    {

        app.MapGet("/getPeople", () =>
personService.GetPeople());

        app.MapGet("/getPerson/{id}", (int id) =>
personService.GetPerson(id));

        app.MapPost("/create", (Person person) =>

        {

            bool result =
personService.Create(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.BadRequest();

        });
```

```
        app.MapPut("/update", (Person person) =>

        {

            bool result =
personService.Update(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

        app.MapDelete("/delete/{id}", (int id) =>

        {

            bool result = personService.Delete(id);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

    }

}
```

PersonEndpoints.cs
Il file contiene la definizione di una classe statica: PersonEndpoints.

Questa classe è quella che deve contenere il livello Controller e quindi la definizione delle API, ma non deve contenere le responsabilità del livello Service e del livello Repository.

Ricordiamo che una classe statica non può essere istanziata e può contenere solo membri statici.

La classe PersonEndpoints ha:

- Un istanza statica di PersonService, che come vedremo è una classe che contiene il livello Service;
- Il metodo statico MapAPIEndpoints, che contiene la definizione delle API e le aggiunge all'app. Dentro le API non troviamo logica. Ogni API utilizza il membro statico personService della classe PersonEndpoints per richiamare i metodi della classe PersonService.

Ogni API della classe PersonEndpoints quindi richiama metodi del Service, prende la risposta e restituisce una risposta al chiamante (client).

Vediamo con la slide successiva la classe PersonService.

Esercitazione9 – CSR

```
public class PersonService

{

    private PersonRepository personRepository = new
PersonRepository();

    public IEnumerable<Person> GetPeople()

    {

        return personRepository.GetPeople();

    }

    public Person? GetPerson(int id)

    {

        return personRepository.GetPerson(id);

    }

    public bool Create(Person person)

    {

        return personRepository.Create(person);

    }

    public bool Update(Person person)

    {

        return personRepository.Update(person);

    }

    public bool Delete(int id)

    {

        return personRepository.Delete(id);

    }

}
```

PersonService.cs
Il file contiene la definizione di una classe: PersonService.

Questa classe è quella che deve contenere il livello Service e quindi la logica aziendale dell'applicativo, ma non deve contenere le responsabilità del livello Cotroller (già definito in PersonEndpoint) e del livello Repository.

La classe PersonService ha:

- Un istanza di PersonRepository, che come vedremo è una classe che contiene il livello Repository;
- La definizione dei metodi richiamati nelle API della classe PersonEndpoints. Questi metodi attualmente non contengono logica, nel nostro caso non stiamo manipolando i dati che ci arrivano in ritorno dal livello Repository, ma avremmo dovuto farlo qui, se necessario. Dentro i metodi non effettuiamo l'accesso diretto ai dati, perché ciò aspetta al livello Repository, infatti ogni metodo della classe PersonService utilizza il membro personRepository della classe PersonService per richiamare i metodi della classe PersonRepository.

Ogni metodo della classe PersonService quindi richiama metodi del Repository, prende la risposta e, dopo aver inserito la logica dell'applicativo, se presente, restituisce una risposta al chiamante (API del livello Controller).

Vediamo con la slide successiva la classe PersonRepository.